narRater

Data Preparation Guide

Where to put each kind of file so the app detects it --
and what you'll see on screen once it does.

You don't need every kind of file -- only the inputs for the steps you run.
A text-only project skips the audio folders; a causal-rating-only project needs just the story events file. Put what you have in the right folder with the right name, and narRater surfaces it automatically. Examples below use the bundled the_siren and pieman_edited data.

How narRater finds your data

Every time you open the app, narRater scans the data/ folders, matches files by their story or subject name and file type, and surfaces what it finds. You never register files by hand -- putting a correctly named file in the correct folder is the whole job.

1. **Put a file in the right data/ folder, named after its story or subject**
2. **narRater scans the folders and matches it by name + file type**
3. **It appears on the Dashboard (a new row / a green-ready cell)**
4. **Open it -- the content is loaded into the inspection view, ready to review/edit**

Matching is forgiving about format, strict about location and name.
Tabular inputs may be .xlsx, .csv, .tsv, .xls, or .txt, and common column-name synonyms are accepted. But the file must live in the folder for its data type and share the story/subject name so narRater can link a recall to its story.

The data folders at a glance

Data type	Put it in	Name it	Format / required columns	Feeds step
Story transcript	`data/2_story_transcript/`	`{story}.txt`	plain UTF-8 text	2 segment
Story events (pre-segmented)	`data/3_story_events/`	`{story}_events.xlsx`	columns: event, story_texts	5 match, 6 rate
Recall text	`data/5_recall_texts/`	`{story}_sub-{id}.txt`	plain UTF-8 text	3 -> 4 -> 5
Story audio (optional)	`data/1_story_audio/`	`{story}.wav`	.wav .mp3 .mp4 .m4a .flac .ogg	1 transcribe
Recall audio (optional)	`data/4_recall_audio/`	`{story}_sub-{id}.mp4`	same audio types	1 transcribe

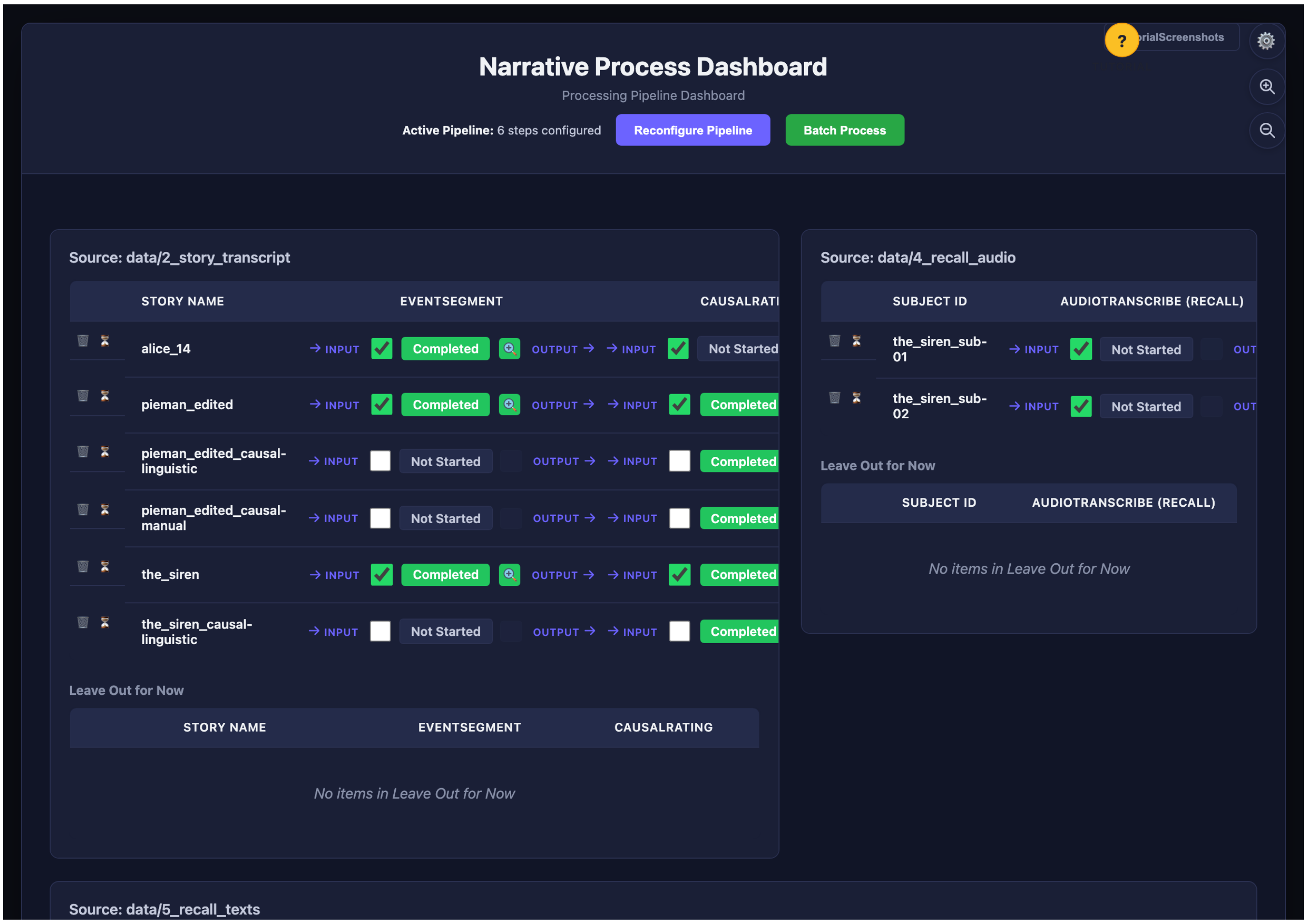
Outputs are written under output/ (one subfolder per step) -- you never put inputs there. The next pages show each input type: where it goes, what it should contain, and what the app displays once it is detected.

Folder map (with the bundled examples)

```
narRaters/
|-- data/                           <- ALL your inputs go here
|    |-- 1_story_audio/             the_siren.wav            (optional, Step 1)
|    |-- 2_story_transcript/        the_siren.txt            (Step 2)
|    |-- 3_story_events/            the_siren_events.xlsx    (Steps 5, 6)
|    |-- 4_recall_audio/            the_siren_sub-01.mp4     (optional, Step 1)
|    |-- 5_recall_texts/            the_siren_sub-01.txt     (Steps 3-5)
|-- output/                         <- results land here automatically
```

What detection looks like: the Dashboard

Open the app and you land on the Dashboard. Each panel is headed by the data/ folder it reads from, every row is a story or subject narRater detected there, and every column is a pipeline step. A grey cell means 'input present, not processed yet'; green means done. If your file is named and placed correctly, its row simply appears here.



The Dashboard. Note the 'Source: data/2_story_transcript', 'Source: data/4_recall_audio' and 'Source: data/5_recall_texts' panel headers -- each lists the items found in that folder. Detected stories/subjects become rows automatically.

1. Story transcript

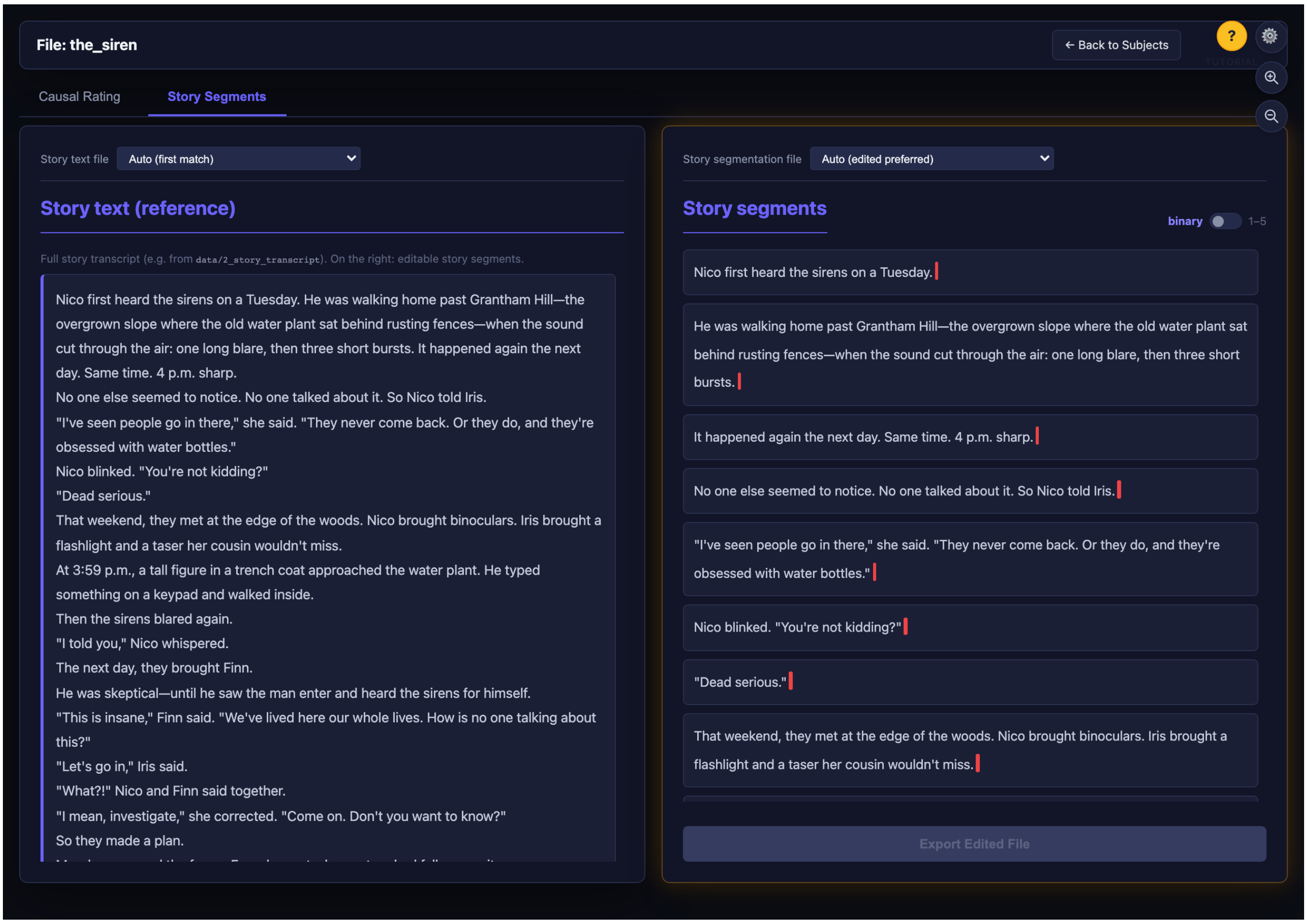
`data/2_story_transcript/` `the_siren.txt`

Plain UTF-8 text, one story per file. Name the file after the story (the same name links its events, audio and recalls). This is the input to Step 2 (segmentation) and the reference text shown throughout the story views.

Example content -- the_siren.txt

```
Nico first heard the sirens on a Tuesday. He was walking home
past Grantham Hill -- the overgrown slope where the old water
plant sat behind rusting fences -- when the sound cut through...
```

What the app shows when detected:



Story Segments tab. The detected transcript loads verbatim into the 'Story text (reference)' panel on the left; the right panel shows it split into numbered events you can edit.

2. Story events (segmented)

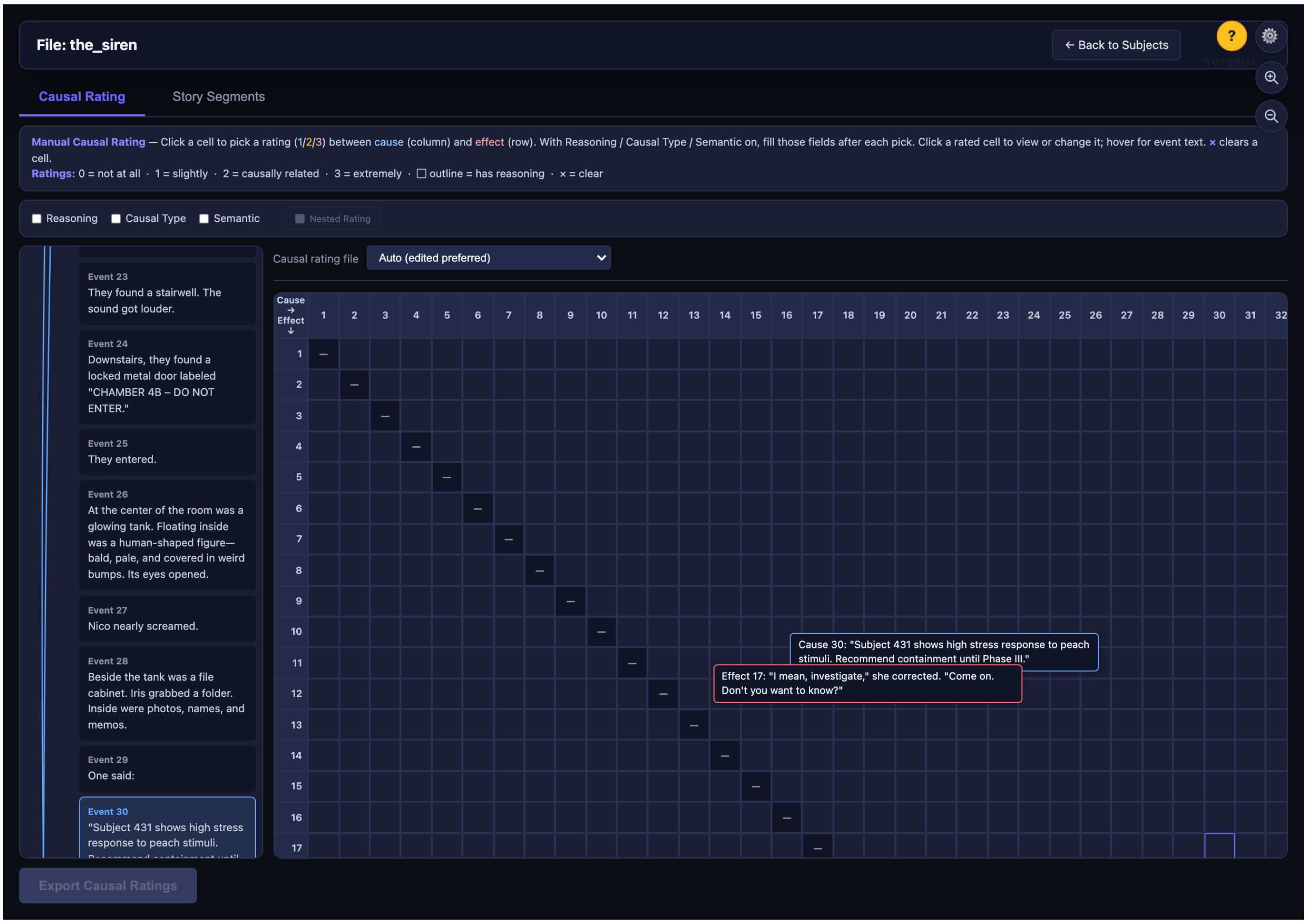
`data/3_story_events/` `the_siren_events.xlsx`

A table with two columns -- event (the number) and story_texts (the segment text). This is what Step 5 matches recalls against and what Step 6 rates for causality. You can drop in a pre-made events file here, or let Step 2 generate one. .csv / .tsv / .xls / .txt also work, and common column synonyms (e.g. 'event_number', 'text') are recognised.

Example content -- the_siren_events.xlsx

```
| event | story_texts                               |
|-------|-------------------------------------------|
| 1     | Nico first heard the sirens on a Tuesday.  |
| 2     | He was walking home past Grantham Hill...  |
| 3     | It happened again the next day. 4 p.m...   |
```

What the app shows when detected:



Causal Rating tab. The detected events become the rows/columns of the cause-by-effect matrix and the numbered list in the left sidebar -- proof the events file was read with its event / story_texts columns.

3. Recall text

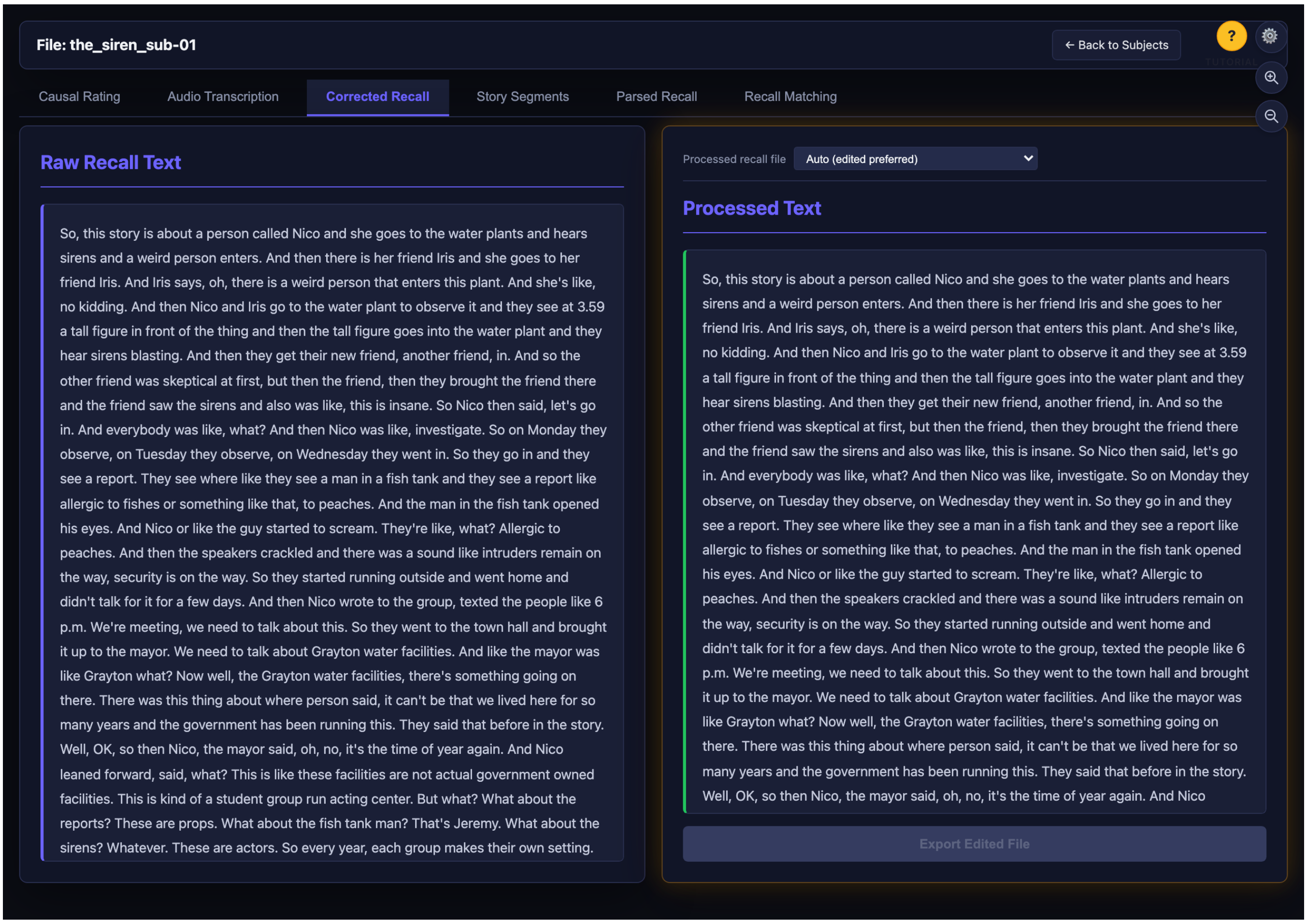
`data/5_recall_texts/` the_siren_sub-01.txt

One subject's recall, transcribed verbatim, as plain UTF-8 text -- one file per subject. Name it {story}_sub-{id} so narRater links the recall to its story (here, the_siren). This feeds Step 3 (spell/grammar), then Step 4 (parsing) and Step 5 (matching).

Example content -- the_siren_sub-01.txt

```
So, this story is about a person called Nico and she goes to
the water plants and hears sirens and a weird person enters.
And then there is her friend Iris and she goes to her friend...
```

What the app shows when detected:



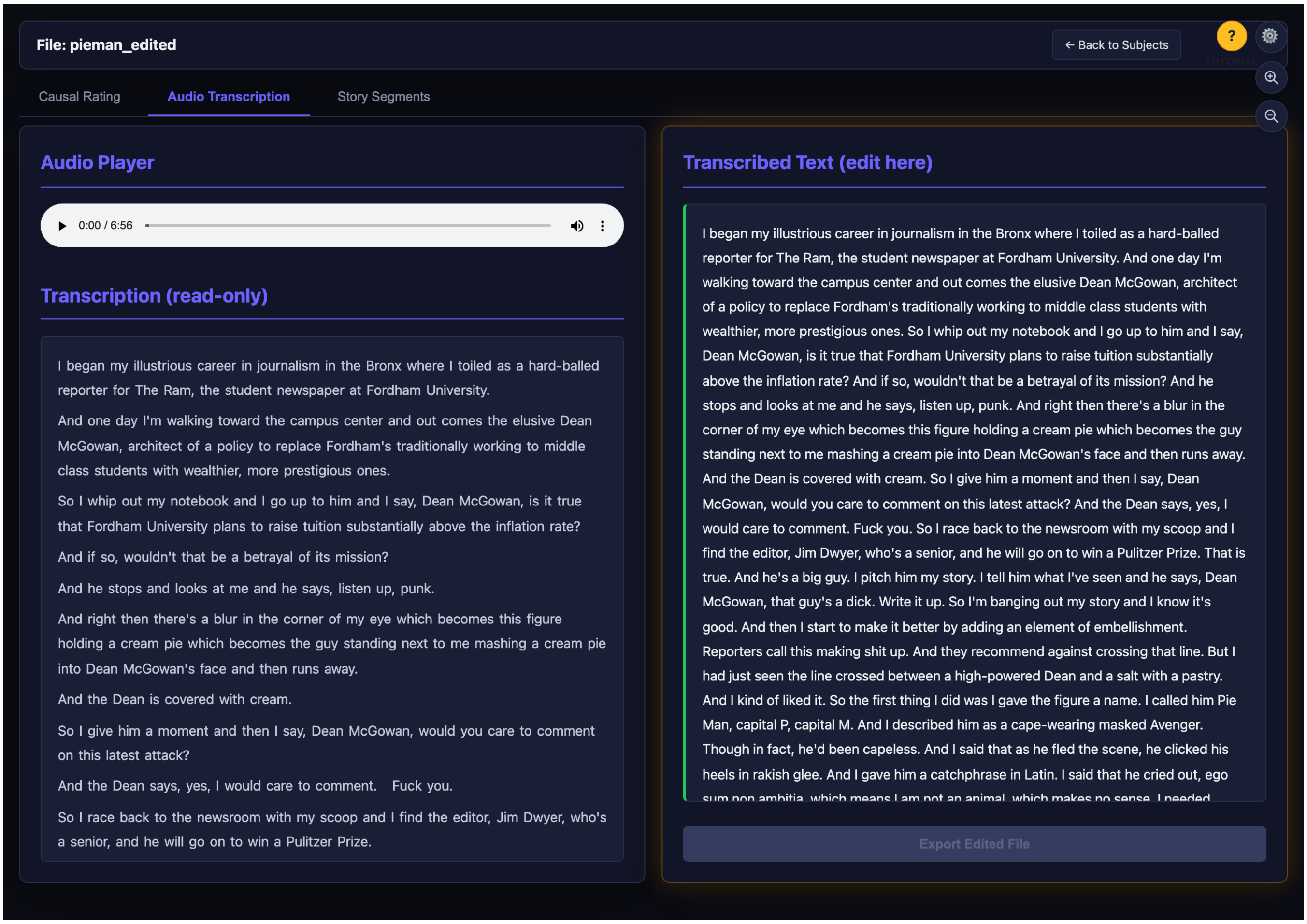
Corrected Recall tab. The detected recall loads into the 'Raw Recall Text' panel on the left (reference); the editable corrected version is on the right. The subject also appears as a row under 'Source: data/5_recall_texts' on the Dashboard.

4. Story audio (optional)

`data/1_story_audio/` `pieman_edited.wav`

Only needed if your story starts as audio rather than text. Accepted types: .wav, .mp3, .mp4, .m4a, .flac, .ogg. Name it after the story. Step 1 transcribes it into output/story_audio-transcribed/{story}.txt.

What the app shows when detected:



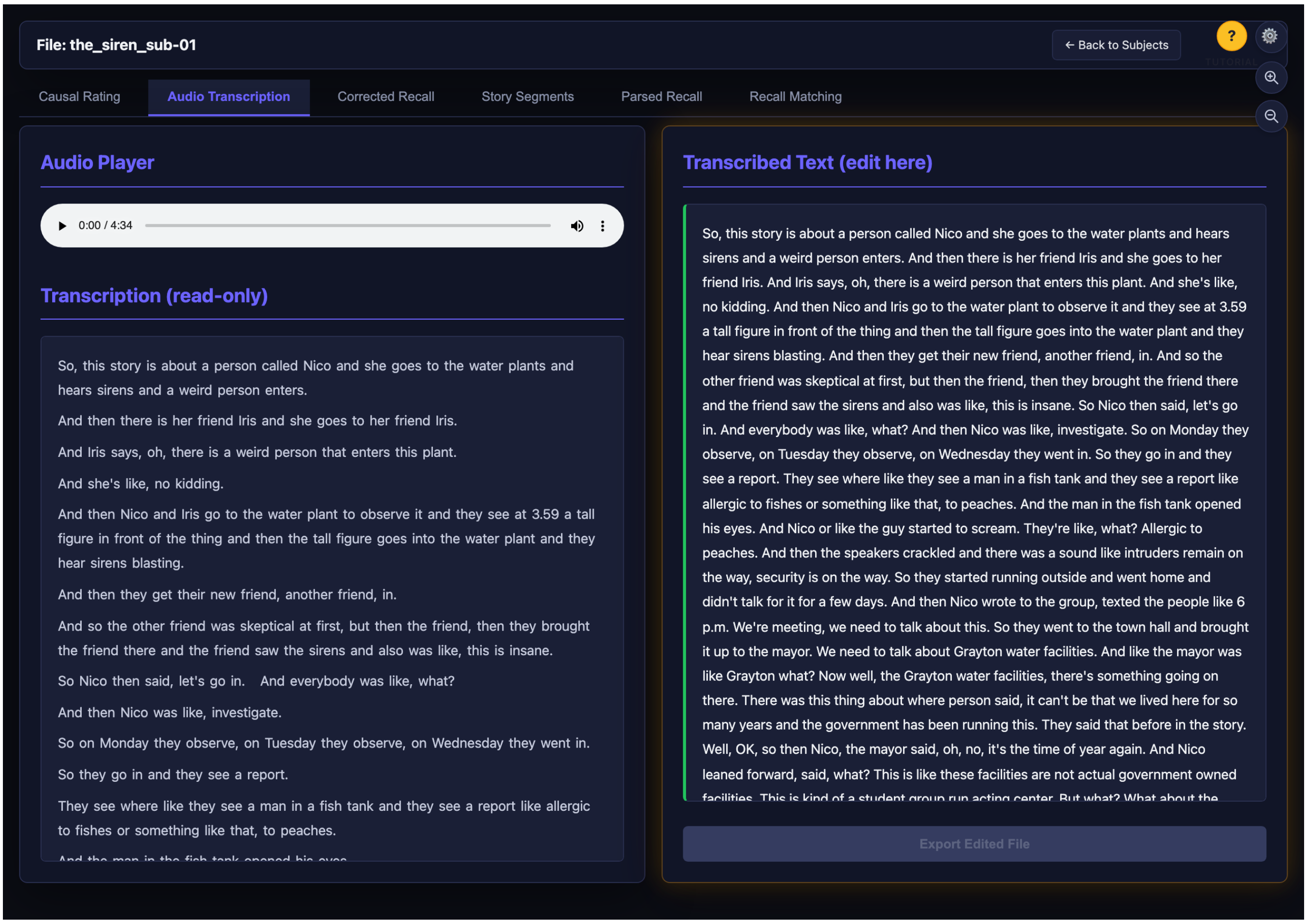
Audio Transcription tab. The detected audio loads into a player (left); after Step 1 runs, the transcript appears on the right, ready to correct by ear.

5. Recall audio (optional)

`data/4_recall_audio/` `the_siren_sub-01.mp4`

Only needed if a subject's recall starts as audio. Same accepted types as story audio. Name it {story}_sub-{id} to match the subject. Step 1 transcribes it into output/recall_audio-transcribed/{story}_sub-{id}.txt, which then flows into Steps 3-5.

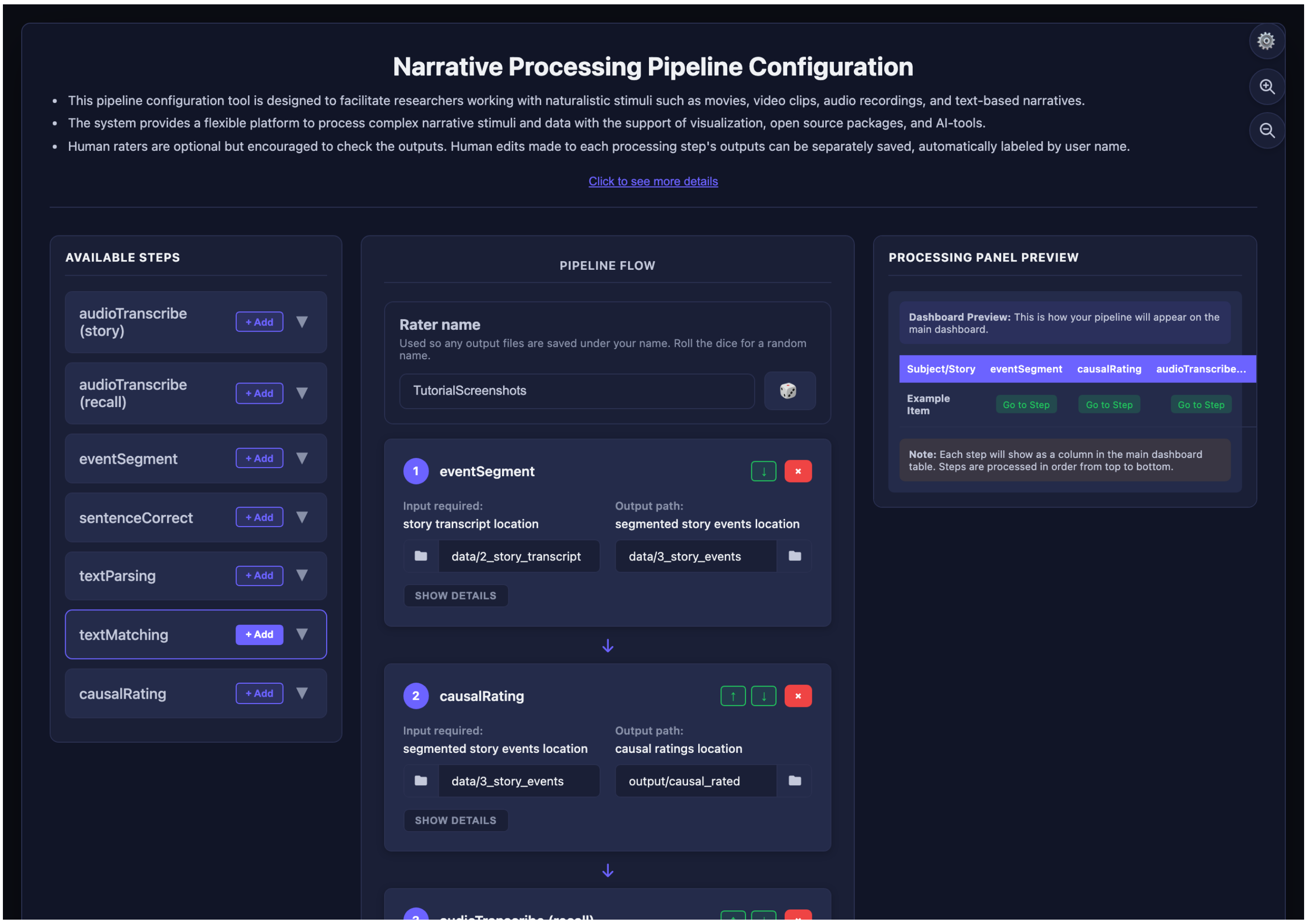
What the app shows when detected:



Audio Transcription tab on a subject. The detected recall recording loads into the player; the editable transcript is on the right.

Where the app looks (and how to change it)

The folders above are the defaults. On the Pipeline Configuration page, every step shows the exact input and output folder it will use, so you can confirm narRater is pointed at your data -- or repoint a step at a different folder without moving files. The two-input Text Matching step shows both an events input and a recall input.



Pipeline Configuration. Each step card shows its Input path (where it reads) and Output path (where it writes); use the folder picker to change either. This is the app telling you exactly where it expects each kind of file.

What successful detection + processing produces

You never place files in output/ -- narRater writes there. Knowing the layout helps you confirm a step succeeded and find files to export.

Step	Writes to	File
1 Transcribe (story)	`output/story_audio-transcribed/`	`{story}.txt`
1 Transcribe (recall)	`output/recall_audio-transcribed/`	`{story}_sub-{id}.txt`
2 Segment	`data/3_story_events/`	`{story}_events-{method}.xlsx`
3 Correct	`output/recall_corrected/`	`{story}_sub-{id}.txt`
4 Parse	`output/recall_parsed/`	`{story}_sub-{id}_parsed.xlsx`
5 Match	`output/recall_rated/`	`{story}_sub-{id}_{method}.xlsx`
6 Causal rate	`output/causal_rated/`	`{story}_causal-{method}.xlsx`

Your hand-edited versions are kept separate.
Automated runs write {id}_{method}.ext; when you edit and export in the browser, narRater saves {id}_{ratername}-edit.ext alongside the original (never overwriting it). A version dropdown in each tab switches between them.

If the app doesn't detect your file

Symptom	Most likely cause / fix
The story/subject row isn't on the Dashboard	File is in the wrong folder, or its extension isn't recognised. Confirm it sits directly in the matching data/ folder (not a sub-folder) with a supported extension.
A recall isn't linked to its story	Name mismatch. Use {story}_sub-{id} (e.g. the_siren_sub-01.txt) so the story name matches the transcript/events file name exactly (case-sensitive).
Events file loads but columns look wrong	Ensure an event-number column and a text column exist (event / story_texts, or accepted synonyms like event_number / text). Swapped columns are auto-corrected on read.
Audio item shows but won't play / transcribe	Use a supported type (.wav .mp3 .mp4 .m4a .flac .ogg) and install the [audio] extra for Step 1.
Nothing appears at all	You may be running from the wrong project root. Start the app from the folder that contains data/ and output/ (or set NARRATERS_PROJECT_ROOT to it).

Still stuck? The README 'Where to put your data' section and narRater_Tutorial.pdf walk through the same folders with more context.